QUANTUM SECURITY INTELLIGENCE

Security & Post-Quantum Readiness Audit Report

fireblocks-mpc-lib-en @ 00ae08b7f1bd · Fireblocks — public repository (passive review)

Organization	QSI
Report date	2026-11-06 13:14 UTC
Report version	1
Audit run	1b8fa6af-00da-4c82-aae7-66504b9373a8 · Document audit · 2026-11-06 13:14 UTC
Analyzer engine	qsi-engine 1.3.0
Risk profile	Crypto exchange, custodian or wallet
Content SHA-256	`23fbd77d28cfee88fe35b18ec32795741bb197e17d217cbdef227c02036a02e3`
Signature key (Ed25519, SHA-256 fingerprint; the signature is the separate file report.sig.json)	`5b3cf9c9bd0fa54447e2b5dfa7a6bab569bd0c666fa46949599e5c348d17083e`

Contents

1. Executive summary
2. Scope and boundaries
3. Methodology
4. Assets examined and not examined
5. Architecture overview
6. Overall security posture
7. Findings by severity
8. Detailed findings
9. Evidence
10. Post-quantum assessment
11. Limitations and assumptions
12. Remediation plan
13. Priorities for 30, 60 and 90 days
14. Verification after remediation
15. Tools used
16. Sources and normative documents
17. Appendices

This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.

Executive summary

7	0	0	7	0
Findings in total	Critical	High	Medium	Low

Classification	Count
Confirmed technical vulnerability	0
Known vulnerable component	0
Confirmed configuration error	0
Potential cryptographic risk	0
Long-term post-quantum risk	7
Improvement recommendation	0
Insufficient data	0

Top 5

ID	Severity	Detailed findings	Priority score
QSI-2026-18011	Medium	Quantum-vulnerable Paillier: threshold signing and MPC protocols (30 file(s)) confirmed by the analyst	41.8
QSI-2026-18012	Medium	Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (8 file(s)) confirmed by the analyst	41.8
QSI-2026-18013	Medium	Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (7 file(s)) confirmed by the analyst	41.8
QSI-2026-18014	Medium	Quantum-vulnerable FROST: threshold signing and MPC protocols (7 file(s)) confirmed by the analyst	41.8
QSI-2026-18015	Medium	Quantum-vulnerable ECDSA: threshold signing and MPC protocols (1 file(s)) confirmed by the analyst	41.8

Mosca's inequality (x + y > z): x = 10, y = 5, z = 8 → migration would finish too late under this scenario. z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Analyst review: 7 confirmed, 0 not applicable, 0 dismissed as false positives.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography. Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Scope and boundaries

Mode: Document audit

Repository: https://github.com/fireblocks/mpc-lib @ 00ae08b7f1bdbb887aedece384aca2f942aae432 — evidence links open the lines at this commit.

Supplied inputs

123 files, 1481254 bytes in total, 1 redaction(s); the per-file SHA-256 list is in appendix C.

Methodology

The audit was performed by the QSI engine 1.3.0. Each analyzer emits facts (rule, parameters, evidence); finding texts are generated from the rule catalog. Methodology sources are listed under "Sources and normative documents".

Prioritization uses an explainable weighted model (severity, exploitability, exposure, confidentiality/integrity/availability impact weighted by the "Crypto exchange, custodian or wallet" profile, asset criticality, urgency, ease of remediation) with a confidence multiplier. CVSS v3.1 is applied only to technical vulnerabilities with a defined vector. Long-term quantum risks use Mosca's inequality; their severity is capped at Medium and they are never presented as exploitable vulnerabilities.

Finding classes: confirmed technical vulnerability; known vulnerable component (a dependency version named in an advisory; whether the affected code is reachable is assessed separately, and a component confirmed by review becomes a vulnerability); confirmed configuration error; potential cryptographic risk; long-term post-quantum risk; improvement recommendation; insufficient data. Heuristic findings are marked preliminary; analyst decisions are shown next to the findings.

Coverage of checks

Area	Status	Count	Examples
source_code	checked	123	include/blockchain/mpc/hd_derive.h: 59 lines scanned with 16 rule patterns and 266 registry patterns; include/cosigner/asymmetric_eddsa_cosigner.h: 54 lines scanned with 16 rule patterns and 266 registry patterns; include/cosigner/asymmetric_eddsa_cosigner_client.h: 51 lines scanned with 16 rule patterns and 266 registry patterns
secrets	checked	1	include/crypto/shamir_secret_sharing/verifiable_secret_sharing.h: 1 redacted value(s) not reported (comments, test modules, fixture declarations, placeholders, values the source annotates as published, or hex values not bound to a secret name)
entropy	checked	1	random-number generation: 123 code files read with 90 macro definitions from the tree; 0 ineffective guard(s), 0 weak random-bytes provider(s), 0 weak key-generation site(s)

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

Assets examined and not examined

Supplied inputs: 124 (input files: C)

Architecture overview

Static review of the source tree fireblocks-mpc-lib-en at 00ae08b7f1bdbb887aedece384aca2f942aae432. 123 files analysed; of 126 files, 1 are test, fuzzing, benchmark or example code and were left out, 2 are not read by any analyzer, 0 exceed the per-file limit and 0 did not fit the total limit. The review was restricted to 3 path(s) of the repository: CMakeLists.txt, include, src; 83 files outside them were not read. Files no analyzer reads include: CMakeLists.txt, src/common/CMakeLists.txt. No requests were sent to the owner's systems.

Overall security posture

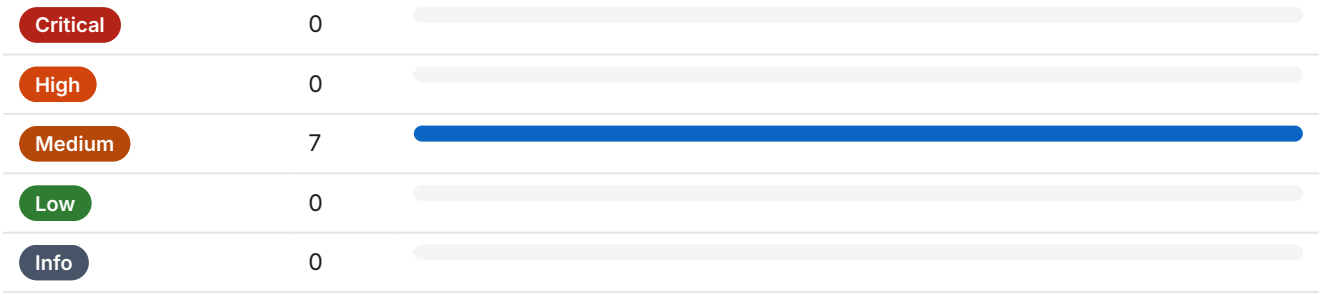
100
Indicative posture index (0–100)

97.8%
Quantum-vulnerable share of inventory

90
Cryptographic inventory

Derived from findings in scope (confirmed and potential issues weighted by severity). Indicative only; not comparable across differently scoped audits.

Findings by severity



ID	Detailed findings	Classification	Severity	Confidence
QSI-2026-18011	Quantum-vulnerable Paillier: threshold signing and MPC protocols (30 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-18012	Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (8 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-18013	Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (7 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-18014	Quantum-vulnerable FROST: threshold signing and MPC protocols (7 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-18015	Quantum-vulnerable ECDSA: threshold signing and MPC protocols (1 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-18017	Quantum-vulnerable Ed25519: threshold signing and MPC protocols (3 file(s))	Long-term post-quantum risk	Medium	Medium
QSI-2026-18016	Quantum-vulnerable Ed25519: role not determined (2 file(s))	Long-term post-quantum risk	Medium	Medium

Detailed findings

QSI-2026-18011 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Paillier: threshold signing and MPC protocols (30 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** The largest row in this repository by reference count — 911 references across 30 files — and the one a post-quantum signature standard does not address. The anchors run through the BAM ECDSA cosigner surface (include/cosigner/bam_ecdsa_cosigner{,_client,_server}.h) and, importantly, through the key persistency structures (bam_key_persistency_structures{,_client,_server}.h): Paillier material is not only used in the protocol, it is *stored*. That matters for sequencing, because stored ciphertext is a harvest-now-decrypt-later item while a protocol is only a future-signing item. Replacing Paillier means redesigning the threshold protocol, not swapping a primitive: NIST's 2024 standards are single-signer, its multi-party threshold call is still open, and the literature notes PQC-round schemes do not convert directly to threshold form through MPC.

911 reference(s) to Paillier across 30 file(s), e.g. include/cosigner/bam_ecdsa_cosigner.h:59. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:Paillier:threshold_mpc
Data source	include/cosigner/bam_ecdsa_cosigner.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:14 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open include/cosigner/bam_ecdsa_cosigner.h:59 and inspect the quoted lines; the evidence lists
further files.
```

EVIDENCE

file_excerpt · include/cosigner/bam_ecdsa_cosigner.h · line 59 · SHA-256 a3d30724e0d1cc72… · File 43a3eaafe7efc587…

```
   58 |         server_setup_shared_data(const T& serialized) :
   59 |             paillier_commitment_pub(serialized.paillier_commitment_pub.begin(),
serialized.paillier_commitment_pub.end()),
   60 |             paillier_blum_zkp(serialized.paillier_blum_zkp.begin(),
serialized.paillier_blum_zkp.end()),
```

file_excerpt · include/cosigner/bam_ecdsa_cosigner.h · line 60 · SHA-256 6be55d1352a2dada… · File 43a3eaafe7efc587…

```
   59 |             paillier_commitment_pub(serialized.paillier_commitment_pub.begin(),
serialized.paillier_commitment_pub.end()),
   60 |             paillier_blum_zkp(serialized.paillier_blum_zkp.begin(),
serialized.paillier_blum_zkp.end()),
   61 |             small_factors_zkp(serialized.small_factors_zkp.begin(),
serialized.small_factors_zkp.end()),
```

file_excerpt · include/cosigner/bam_ecdsa_cosigner.h · line 68 · SHA-256 13aec856ddc4a274… · File 43a3eaafe7efc587…

```
   67 |         //paillier commitment key and proofs
   68 |         byte_vector_t paillier_commitment_pub;
   69 |         byte_vector_t paillier_blum_zkp;
```

file_excerpt · include/cosigner/bam_ecdsa_cosigner_client.h · line 118 · SHA-256 f87b02773d914f66… · File bbb3628a1a863822…

```
  117 |
  118 |     void well_formed_signature_range_proof_generate(const paillier_commitment_public_key_t
*paillier,
  119 |                                                     elliptic_curve256_algebra_ctx* algebra,
```

file_excerpt · include/cosigner/bam_ecdsa_cosigner_server.h · line 97 · SHA-256 27067a51e815109b… · File 1f878ae3e3f65f63…

```
   96 |                               const byte_vector_t& setup_aad,
   97 |                               byte_vector_t& paillier_blum,
   98 |                               byte_vector_t& small_factors,
```

file_excerpt · include/cosigner/bam_key_persistency_structures_client.h · line 5 · SHA-256 395957d01b864c5b… · File 7cbd61de453cb52f…

```
    4 |
    5 | #include "crypto/paillier_commitment/paillier_commitment.h"
    6 | #include "crypto/commitments/damgard_fujisaki.h"
```

file_excerpt · include/cosigner/bam_key_persistency_structures_client.h · line 28 · SHA-256 5d17cd62f0c574e2… · File 7cbd61de453cb52f…

```
   27 |     }
   28 |     std::shared_ptr<paillier_commitment_public_key_t> paillier_commitment_pub;
   29 | };
```

file_excerpt · include/cosigner/bam_key_persistency_structures_server.h · line 4 · SHA-256 0160e73aad94c69f… · File 68e9a88bb24c8408…

```
    3 | #include "bam_key_persistency_structures.h"
    4 | #include "crypto/paillier_commitment/paillier_commitment.h"
    5 | #include "crypto/commitments/damgard_fujisaki.h"
```

file_excerpt · include/cosigner/bam_key_persistency_structures_server.h · line 16 · SHA-256 d6f03415a6648f23… · File 68e9a88bb24c8408…

```
   15 |     bam_setup_auxilary_key_server() = default;
   16 |     std::shared_ptr<paillier_commitment_private_key_t> paillier_commitment_priv;
   17 | };
```

file_excerpt · include/cosigner/cmp_ecdsa_signing_service.h · line 7 · SHA-256 c4c325a0537c146a… · File 5f9cb66f067166d1…

```
    6 | #include "cosigner/cosigner_exception.h"
    7 | #include "crypto/paillier/paillier.h"
    8 |
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18012 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Pedersen commitment: threshold signing and MPC protocols (8 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Correct, anchored on a real implementation (include/crypto/commitments/pedersen.h) plus the cosigner clients and servers that use it and the key persistency structures that store it. Stated precisely: breaking the discrete log costs a Pedersen commitment its binding property, not its hiding — it is perfectly hiding, so committed values are not recoverable even with a CRQC. The exposure is forgeability within the proofs, which is future soundness rather than retroactive disclosure. Do not describe this row as "amounts become readable".

27 reference(s) to Pedersen commitment across 8 file(s), e.g. include/cosigner/bam_ecdsa_cosigner_client.h:8. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:Pedersen commitment:threshold_mpc
Data source	include/cosigner/bam_ecdsa_cosigner_client.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:14 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open include/cosigner/bam_ecdsa_cosigner_client.h:8 and inspect the quoted lines; the evidence lists
further files.
```

EVIDENCE

file_excerpt · include/cosigner/bam_ecdsa_cosigner_client.h · line 8 · SHA-256 9d56e11fcd768b11... · File bbb3628a1a863822...

```
    7 | struct elliptic_curve256_algebra_ctx;
    8 | struct pedersen_commitment_two_generators;
    9 |
```

file_excerpt · include/cosigner/bam_ecdsa_cosigner_client.h · line 120 · SHA-256 45aa0403cca9b7fa... · File bbb3628a1a863822...

```
  119 |                                                 elliptic_curve256_algebra_ctx* algebra,
  120 |                                                 const
pedersen_commitment_two_generators* ec_base,
  121 |                                                 const commitments_sha256_t&
signature_add,
```

file_excerpt · include/cosigner/bam_key_persistency_structures.h · line 20 · SHA-256 97396fe65efec088... · File b8f440b5865c6694...

```
   19 |     //algorithm is required because because ec_base is points on a specific curve
   20 |     pedersen_commitment_two_generators_t ec_base { {0}, {0} };
   21 |     cosigner_sign_algorithm setup_algorithm{(cosigner_sign_algorithm)-1};
```

file_excerpt · include/crypto/commitments/pedersen.h · line 25 · SHA-256 b4288862b6ee842d... · File 64f9b6334199aea5...

```
   24 |  */
   25 | typedef struct pedersen_commitment_two_generators
   26 | {
```

file_excerpt · include/crypto/commitments/pedersen.h · line 29 · SHA-256 c382ef08fb01911e... · File 64f9b6334199aea5...

```
   28 |     elliptic_curve256_point_t f;
   29 | } pedersen_commitment_two_generators_t;
   30 |
```

file_excerpt · include/crypto/commitments/pedersen.h · line 44 · SHA-256 c46469dcb7763521... · File 64f9b6334199aea5...

```
   43 |  */
   44 | COSIGNER_EXPORT commitments_status
pedersen_commitment_two_generators_base_generate(pedersen_commitment_two_generators_t* base,
   45 |                                                                                 const
uint8_t* aad,
```

file_excerpt · src/common/cosigner/bam_ecdsa_cosigner_client.cpp · line 193 · SHA-256 926c08a32756c401... · File d5e5a60b90b9d16e...

```
  192 |     const byte_vector_t setup_aad = generate_setup_aad_bytes(setup_id,
client_key_metadata.algorithm);
  193 |     commitments_status com_status =
pedersen_commitment_two_generators_base_generate(&client_key_metadata.ec_base,
  194 |
setup_aad.data(),
```

file_excerpt · src/common/cosigner/bam_ecdsa_cosigner_client.cpp · line 448 · SHA-256 7882963c7c7f8c7e... · File d5e5a60b90b9d16e...

```
  447 |
elliptic_curve256_algebra_ctx *algebra,
  448 |                                                                    const
pedersen_commitment_two_generators *ec_base, //two points h and f
  449 |                                                                    const
commitments_sha256_t& signature_aad,
```

file_excerpt · src/common/cosigner/bam_ecdsa_cosigner_server.cpp · line 104 · SHA-256 f67b3d1bf5695c08… · File dfa91f27d28fe424…

```
  103 |     // Generate base commitment
  104 |     const commitments_status com_status =
pedersen_commitment_two_generators_base_generate(&server_setup_metadata.ec_base,
  105 |
setup_aad.data(),
```

file_excerpt · src/common/cosigner/bam_well_formed_proof.cpp · line 167 · SHA-256 82c45265099193cf… · File e7393aba28ea9797…

```
  166 |                              const commitments_sha256_t& signature_aad,
  167 |                              const pedersen_commitment_two_generators_t& ec_base,
  168 |                              const paillier_commitment_public_key_t& paillier,
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18013 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Threshold ECDSA: threshold signing and MPC protocols (7 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Correct and well anchored, unlike the equivalent row in cb-mpc: the files are the CMP signing services themselves (include/cosigner/cmp_ecdsa_{offline,online,}_signing_service.h and src/common/cosigner/cmp_ecdsa_offline_signing_service.cpp), i.e. an implementation of the CMP/CGGMP threshold ECDSA protocol. This is the core custody operation: every transaction the cosigner authorises goes through it.

43 reference(s) to Threshold ECDSA across 7 file(s), e.g. include/cosigner/cmp_ecdsa_offline_signing_service.h:5. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:Threshold ECDSA:threshold_mpc
Data source	include/cosigner/cmp_ecdsa_offline_signing_service.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:14 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open include/cosigner/cmp_ecdsa_offline_signing_service.h:5 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · include/cosigner/cmp_ecdsa_offline_signing_service.h · line 5 · SHA-256 5d5a2072dbd88f50… · File 5f6e2ebfef941b6d…

```
    4 |
    5 | #include "cosigner/cmp_ecdsa_signing_service.h"
    6 |
```

file_excerpt · include/cosigner/cmp_ecdsa_offline_signing_service.h · line 17 · SHA-256 1b5adf8d72144642… · File 5f6e2ebfef941b6d…

```
   16 | // this class implements MPC CMP for offline signing based on https://eprint.iacr.org/2020/492 paper
   17 | class COSIGNER_EXPORT cmp_ecdsa_offline_signing_service final : public cmp_ecdsa_signing_service
   18 | {
```

file_excerpt · include/cosigner/cmp_ecdsa_offline_signing_service.h · line 41 · SHA-256 de949e6058939704… · File 5f6e2ebfef941b6d…

```
   40 |
   41 |     cmp_ecdsa_offline_signing_service(platform_service& service, const cmp_key_persistency& key_persistency, preprocessing_persistency& persistency) : cmp_ecdsa_signing_service(service, key_persistency), _preprocessing_persistency(persistency) {}
   42 |
```

file_excerpt · include/cosigner/cmp_ecdsa_online_signing_service.h · line 5 · SHA-256 f4205c3420ea3979… · File 88a9477e41b6be37…

```
    4 |
    5 | #include "cosigner/cmp_ecdsa_signing_service.h"
    6 | #include "cosigner/timing_map.h"
```

file_excerpt · include/cosigner/cmp_ecdsa_online_signing_service.h · line 34 · SHA-256 dee2ab8850981985… · File 88a9477e41b6be37…

```
   33 | // this class implements MPC CMP for online signing based on https://eprint.iacr.org/
2020/492 paper
   34 | class COSIGNER_EXPORT cmp_ecdsa_online_signing_service final : public
cmp_ecdsa_signing_service
   35 | {
```

file_excerpt · include/cosigner/cmp_ecdsa_online_signing_service.h · line 48 · SHA-256 5ca1c386ac827ef9… · File 88a9477e41b6be37…

```
   47 |
   48 |     cmp_ecdsa_online_signing_service(platform_service& service, const cmp_key_persistency&
key_persistency, signing_persistency& persistency);
   49 |
```

file_excerpt · include/cosigner/cmp_ecdsa_signing_service.h · line 90 · SHA-256 bfd7c0125f4d8f0b… · File 5f9cb66f067166d1…

```
   89 | // this class holds the common functionality for cmp_ecdsa_online_signing_service and
cmp_ecdsa_offline_signing_service
   90 | class COSIGNER_EXPORT cmp_ecdsa_signing_service
   91 | {
```

file_excerpt · include/cosigner/cmp_ecdsa_signing_service.h · line 93 · SHA-256 fe23a1ba2f04d34d… · File 5f9cb66f067166d1…

```
   92 | protected:
   93 |     cmp_ecdsa_signing_service(platform_service& service, const cmp_key_persistency&
key_persistency) : _service(service), _key_persistency(key_persistency) {}
   94 |     virtual ~cmp_ecdsa_signing_service();
```

file_excerpt · include/cosigner/cmp_ecdsa_signing_service.h · line 94 · SHA-256 25164deb5b15a258… · File 5f9cb66f067166d1…

```
   93 |     cmp_ecdsa_signing_service(platform_service& service, const cmp_key_persistency&
key_persistency) : _service(service), _key_persistency(key_persistency) {}
   94 |     virtual ~cmp_ecdsa_signing_service();
   95 |
```

file_excerpt · src/common/cosigner/cmp_ecdsa_offline_signing_service.cpp · line 1 · SHA-256 bbf5a7f92ba6fdcd… · File 29718caacb308f10…

```
    1 | #include "cosigner/cmp_ecdsa_offline_signing_service.h"
    2 | #include "cosigner/cmp_signature_preprocessed_data.h"
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18014 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable FROST: threshold signing and MPC protocols (7 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Correct: include/cosigner/frost_cosigner{,_client,_server}.h and platform_service.h, 70 references. FROST is a Schnorr-based threshold signature over a prime-order group, so Shor breaks it by the same discrete-log attack as the ECDSA protocols — worth a separate row because FROST is often presented as the modern, safer choice, and its quantum exposure is identical. There is no post-quantum FROST analogue in production.

70 reference(s) to FROST across 7 file(s), e.g. include/cosigner/frost_cosigner.h:17. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:FROST:threshold_mpc
Data source	include/cosigner/frost_cosigner.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:14 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open include/cosigner/frost_cosigner.h:17 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · include/cosigner/frost_cosigner.h · line 17 · SHA-256 dfb2f2409ba54ee9… · File cdfdfc52b15629df…

```
16 |
17 | struct frost_signing_properties
18 | {
```

file_excerpt · include/cosigner/frost_cosigner.h · line 22 · SHA-256 27e7eab089c30bb8… · File cdfdfc52b15629df…

```
   21 |
   22 | class COSIGNER_EXPORT frost_cosigner
   23 | {
```

file_excerpt · include/cosigner/frost_cosigner.h · line 68 · SHA-256 10e549016afff222… · File cdfdfc52b15629df…

```
   67 |
   68 |     explicit frost_cosigner(platform_service& platform_service);
   69 |
```

file_excerpt · include/cosigner/frost_cosigner_client.h · line 3 · SHA-256 7f634f4c985a8e36… · File 1fff442d632f1c26…

```
    2 |
    3 | #include "cosigner/frost_cosigner.h"
    4 | #include <vector>
```

file_excerpt · include/cosigner/frost_cosigner_client.h · line 11 · SHA-256 36ee4725d41505f6… · File 1fff442d632f1c26…

```
   10 |
   11 | class COSIGNER_EXPORT frost_cosigner_client : public frost_cosigner
   12 | {
```

file_excerpt · include/cosigner/frost_cosigner_client.h · line 14 · SHA-256 ecba6ee6af3bbfc8… · File 1fff442d632f1c26…

```
   13 | public:
   14 |     frost_cosigner_client(platform_service& platform_service, frost_key_persistency_client&
key_persistency);
   15 |     virtual ~frost_cosigner_client() = default;
```

file_excerpt · include/cosigner/frost_cosigner_server.h · line 3 · SHA-256 4b2fa08b15d5c1b1… · File 423f364293a16ec6…

```
    2 |
    3 | #include "cosigner/frost_cosigner.h"
    4 | #include <set>
```

file_excerpt · include/cosigner/frost_cosigner_server.h · line 13 · SHA-256 8fe44e7e442644be… · File 423f364293a16ec6…

```
   12 |
   13 | class COSIGNER_EXPORT frost_cosigner_server : public frost_cosigner
   14 | {
```

file_excerpt · include/cosigner/frost_cosigner_server.h · line 16 · SHA-256 4f29d89dfa98bac9… · File 423f364293a16ec6…

```
   15 | public:
   16 |     frost_cosigner_server(platform_service& platform_service, frost_key_persistency_server&
key_persistency, frost_tx_persistency& tx_persistency);
   17 |     virtual ~frost_cosigner_server() = default;
```

file_excerpt · include/cosigner/platform_service.h · line 20 · SHA-256 695e178fa96f0da1… · File 9c7e53a65dd5bc6b…

```
   19 | struct bam_signing_properties;
   20 | struct frost_signing_properties;
   21 |
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18015 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable ECDSA: threshold signing and MPC protocols (1 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** include/cosigner/sign_algorithm.h:10 — the enumeration of supported signing algorithms. A small row, and the right one to look at first in a migration: the place where a new scheme has to be admitted into the protocol's type system, and therefore a version-compatibility problem across every deployed cosigner.

2 reference(s) to ECDSA across 1 file(s), e.g. include/cosigner/sign_algorithm.h:10. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:ECDSA:threshold_mpc
Data source	include/cosigner/sign_algorithm.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:14 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open include/cosigner/sign_algorithm.h:10 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · include/cosigner/sign_algorithm.h · line 10 · SHA-256 7352349ce44da6c4… · File 99f94f61f6a92527…

```
 9 |     ECDSA_STARK,
10 |     SCHNORR_SECP256K1, // Schnorr signature on secp256k1
11 |     SCHNORR_SECP256R1, // Schnorr signature on secp256r1
```

file_excerpt · include/cosigner/sign_algorithm.h · line 11 · SHA-256 42921b9b13eaa15a... · File 99f94f61f6a92527...

```
10 |     SCHNORR_SECP256K1, // Schnorr signature on secp256k1
11 |     SCHNORR_SECP256R1, // Schnorr signature on secp256r1
12 |     SCHNORR_STARK      // Schnorr signature on stark
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18017 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Ed25519: threshold signing and MPC protocols (3 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Threshold EdDSA signing: src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp:401, eddsa_online_signing_service.cpp and frost_cosigner_server.cpp. Same migration class as the FROST and threshold-ECDSA rows.

4 reference(s) to Ed25519 across 3 file(s), e.g. src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp:401. Role, judged from paths, function names and nearby code: threshold signing and MPC protocols. Paillier, Pedersen commitments and the discrete-logarithm proofs inside threshold ECDSA and EdDSA protocols all fall to Shor's algorithm; a post-quantum threshold scheme is a protocol change, not a parameter change.

Affected asset	source-code:Ed25519:threshold_mpc
Data source	src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:14 UTC
Priority score	41.8 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp:401 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp · line 401 · SHA-256 6e04b3a3cf25de44… · File 15e49277a9de05d8…

```
  400 |             memcpy(raw_sig + 32, sig.s, 32);
  401 |             if (ed25519_verify(ed25519, (const uint8_t*)data.sig_data[i].message.data(),
data.sig_data[i].message.size(), raw_sig, derived_public_key, data.sig_data[i].flags &
EDDSA_KECCAK))
  402 |             {
```

file_excerpt · src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp · line 497 · SHA-256 8f5dbe923034531e… · File 15e49277a9de05d8…

```
  496 |         memcpy(raw_sig + 32, cur_sig.s, 32);
  497 |         if (ed25519_verify(ed25519, (const uint8_t*)data.sig_data[index].message.data(),
data.sig_data[index].message.size(), raw_sig, derived_public_key, data.sig_data[index].flags &
EDDSA_KECCAK))
  498 |         {
```

file_excerpt · src/common/cosigner/eddsa_online_signing_service.cpp · line 382 · SHA-256 d641b422ac935789… · File e8264a8e3e84d092…

```
  381 |         memcpy(raw_sig + 32, cur_sig.s, 32);
  382 |         if (ed25519_verify(ed25519, (const uint8_t*)data.sig_data[index].message.data(),
data.sig_data[index].message.size(), raw_sig, derived_public_key, data.sig_data[index].flags &
EDDSA_KECCAK))
  383 |         {
```

file_excerpt · src/common/cosigner/frost_cosigner_server.cpp · line 65 · SHA-256 2a70a07e1414d949… · File a356cd59f4b11d71…

```
   64 |         memcpy(raw_sig + 32, cur_sig.s, 32);
   65 |         return ed25519_verify((ed25519_algebra_ctx_t *)ctx->ctx, message, message_len,
raw_sig, *derived_public_key, use_keccak);
   66 |     }
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

QSI-2026-18016 · CODE-QV-ASYMMETRIC

Medium

Quantum-vulnerable Ed25519: role not determined (2 file(s))

Long-term post-quantum risk | Confidence: Medium

> **Analyst review (QSI review, 2026-10-28): confirmed by the analyst.** Role resolved: both anchors are include/crypto/ed25519_algebra/ed25519_algebra.h:71 and src/common/crypto/ed25519_algebra/ed25519_algebra.c — the Ed25519 group and field arithmetic layer that the EdDSA and FROST cosigners are built on, not an independent key use. It belongs in the inventory because a post-quantum scheme replaces this layer wholesale, but it is the same exposure as the threshold EdDSA rows reached one level lower, so it must not be counted as additional surface.

2 reference(s) to Ed25519 across 2 file(s), e.g. include/crypto/ed25519_algebra/ed25519_algebra.h:71. Role, judged from paths, function names and nearby code: role not determined. The code does not show what the keys protect; review decides the priority.

Affected asset	source-code:Ed25519:general
Data source	include/crypto/ed25519_algebra/ed25519_algebra.h
Detection method	Regular-expression matching of crypto API patterns; the role comes from words in paths, enclosing function names and the lines around the matches.
Analyzer	source_code 1.3.0 · 2026-11-06 13:14 UTC
Priority score	40.2 / 100 (Crypto exchange, custodian or wallet)
CVSS v3.1	not applicable
Mosca's inequality (x + y > z)	x=10.0, y=5.0, z=8.0 → migration would finish too late under this scenario

POTENTIAL IMPACT

Long-term risk depends on what the algorithm protects: signatures (authenticity until migration) or key exchange/encryption (retroactive decryption of recorded data).

REPRODUCTION STEPS

```
Open include/crypto/ed25519_algebra/ed25519_algebra.h:71 and inspect the quoted lines; the evidence lists further files.
```

EVIDENCE

file_excerpt · include/crypto/ed25519_algebra/ed25519_algebra.h · line 71 · SHA-256 5d36dd9af9202966… · File f2e14ae6c9654a17…

```
   70 | /* Verifies the signature using the message and public_key key */
   71 | COSIGNER_EXPORT int ed25519_verify(const ed25519_algebra_ctx_t *ctx, const uint8_t *message,
size_t message_len, const uint8_t signature[64], const uint8_t public_key[32], uint8_t use_keccak);
   72 | #ifdef __cplusplus
```

file_excerpt · src/common/crypto/ed25519_algebra/ed25519_algebra.c · line 671 · SHA-256 e6eb5ecf7d335902… · File f772f412431b5cd1…

```
670 |
671 | int ed25519_verify(const ed25519_algebra_ctx_t *ctx,
672 |                    const uint8_t *message,
```

LIMITATIONS OF THE CHECK

This is a forward-looking risk that depends on the arrival of a cryptographically relevant quantum computer (CRQC). No such computer is publicly known to exist today; the timeline is uncertain. It is not a present-day exploitable vulnerability. Heuristic pattern match on source text without data-flow analysis. The usage context (security-relevant or not) must be confirmed manually; treat as preliminary.

RECOMMENDATION

Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility.

RETEST PROCEDURE

Re-run the inventory after refactoring.

P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997); NIST IR 8547 — Transition to Post-Quantum Cryptography Standards; M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018)

Evidence

ID	Type	Data source	SHA-256
QSI-2026-18011	file_excerpt	include/cosigner/bam_ecdsa_cosigner.h (line 59)	a3d30724e0d1cc720bdcee1ab854050dd97aae791a7595
QSI-2026-18011	file_excerpt	include/cosigner/bam_ecdsa_cosigner.h (line 60)	6be55d1352a2dadaba5b8b748068c0b73fc7c40c84c9cd
QSI-2026-18011	file_excerpt	include/cosigner/bam_ecdsa_cosigner.h (line 68)	13aec856ddc4a2742afb62c3e239209953f48d8c98885b
QSI-2026-18011	file_excerpt	include/cosigner/bam_ecdsa_cosigner_client.h (line 118)	f87b02773d914f663d4646cfbba89121eafab8e9c02ebf
QSI-2026-18011	file_excerpt	include/cosigner/bam_ecdsa_cosigner_server.h (line 97)	27067a51e815109bc25c793e51c5d24aede4300c25a6da
QSI-2026-18011	file_excerpt	include/cosigner/bam_key_persistency_structures_client.h (line 5)	395957d01b864c5bd185058af22e6930049851cbd1e6ae
QSI-2026-18011	file_excerpt	include/cosigner/bam_key_persistency_structures_client.h (line 28)	5d17cd62f0c574e2b2f1a94e2f78db87e6734ef470dff3
QSI-2026-18011	file_excerpt	include/cosigner/bam_key_persistency_structures_server.h (line 4)	0160e73aad94c69f987099154af5b60cceee8d9d5181eb
QSI-2026-18011	file_excerpt	include/cosigner/bam_key_persistency_structures_server.h (line 16)	d6f03415a6648f23561345d5b5fea2159d25e656c36b6f
QSI-2026-18011	file_excerpt	include/cosigner/cmp_ecdsa_signing_service.h (line 7)	c4c325a0537c146aa80b2ea30c08fa653773649c24c2ea
QSI-2026-18012	file_excerpt	include/cosigner/bam_ecdsa_cosigner_client.h (line 8)	9d56e11fcd768b11849b71756b1600de96403ccc80d0f0
QSI-2026-18012	file_excerpt	include/cosigner/bam_ecdsa_cosigner_client.h (line 120)	45aa0403cca9b7fa8cc248f5d1f7ecb002b5931261882d
QSI-2026-18012	file_excerpt	include/cosigner/bam_key_persistency_structures.h (line 20)	97396fe65efec08897595aa1688eadc04c0455a50d82b4
QSI-2026-18012	file_excerpt	include/crypto/commitments/pedersen.h (line 25)	b4288862b6ee842d39e8fa2e3d914c30eace3e9e2350b0
QSI-2026-18012	file_excerpt	include/crypto/commitments/pedersen.h (line 29)	c382ef08fb01911ed7ae5b82f5d9e91c6af09a18a7fc1a
QSI-2026-18012	file_excerpt	include/crypto/commitments/pedersen.h (line 44)	c46469dcb7763521d974aa9032ad397fd4b3739244adf4
QSI-2026-18012	file_excerpt	src/common/cosigner/bam_ecdsa_cosigner_client.cpp (line 193)	926c08a32756c401a76a7ac45e4f3348475af84b60a0d4

QSI-2026-18012	file_excerpt	src/common/cosigner/bam_ecdsa_cosigner_client.cpp (line 448)	7882963c7c7f8c7eb0be6fe7f3b6486898db202092a55a
QSI-2026-18012	file_excerpt	src/common/cosigner/bam_ecdsa_cosigner_server.cpp (line 104)	f67b3d1bf5695c08b42ba40216c5c41bf8db4a2e207a15
QSI-2026-18012	file_excerpt	src/common/cosigner/bam_well_formed_proof.cpp (line 167)	82c45265099193cf93bfa2b7a5c7e63ed91ed5b2efc2c4
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_offline_signing_service.h (line 5)	5d5a2072dbd88f509f650bb4bc0446ff74dc396986e369
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_offline_signing_service.h (line 17)	1b5adf8d721446429fa3f1a9577872e40f6c67cb9e9ba8
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_offline_signing_service.h (line 41)	de949e60589397048d1a0c7cef04c62f2932210074f171
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_online_signing_service.h (line 5)	f4205c3420ea3979b75a127cf72d4b949b9147c13c1b47
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_online_signing_service.h (line 34)	dee2ab8850981985bfdf4c16f5f5d12b130a8c509e89f2
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_online_signing_service.h (line 48)	5ca1c386ac827ef9aab7b179add7e26d2de76ee53eefe3
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_signing_service.h (line 90)	bfd7c0125f4d8f0be9349304cdef8653469765c8de87ff
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_signing_service.h (line 93)	fe23a1ba2f04d34df7f16a613d5a3fe361f4801518c6b8
QSI-2026-18013	file_excerpt	include/cosigner/cmp_ecdsa_signing_service.h (line 94)	25164deb5b15a258b18482a7e9c6dd78cb34bff1dabcce
QSI-2026-18013	file_excerpt	src/common/cosigner/cmp_ecdsa_offline_signing_service.cpp (line 1)	bbf5a7f92ba6fdcdb5e3fbd07ed1dd31b4d507c8895f94
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner.h (line 17)	dfb2f2409ba54ee92edb786b2df6cd4951b60ef2e88d67
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner.h (line 22)	27e7eab089c30bb8844d3c4608c16973c9849ac4f27d80
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner.h (line 68)	10e549016afff2225eb878fb7a3ace11b8fd27bbd59f3a
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner_client.h (line 3)	7f634f4c985a8e3630c7ba6f31a04f3e30f4521a6596c9

QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner_client.h (line 11)	36ee4725d41505f6e6e6b590bcb911cc0e4600d890419f
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner_client.h (line 14)	ecba6ee6af3bbfc8f96475a6d5dc031b9648949a55f357
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner_server.h (line 3)	4b2fa08b15d5c1b128a7c27b0cdca54af1f28727e21745
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner_server.h (line 13)	8fe44e7e442644bec7724a4252325bebfed2dae8007387
QSI-2026-18014	file_excerpt	include/cosigner/frost_cosigner_server.h (line 16)	4f29d89dfa98bac9f5f7dc7b07f7a50dcf452b03d37fc4
QSI-2026-18014	file_excerpt	include/cosigner/platform_service.h (line 20)	695e178fa96f0da1c35a5b8e52904e0224592499365aa5
QSI-2026-18015	file_excerpt	include/cosigner/sign_algorithm.h (line 10)	7352349ce44da6c46a0059400e65ebf3677bd6b31dd579
QSI-2026-18015	file_excerpt	include/cosigner/sign_algorithm.h (line 11)	42921b9b13eaa15a54bcf54577536b5f909c8ee17a9903
QSI-2026-18017	file_excerpt	src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp (line 401)	6e04b3a3cf25de443120e16f8ddcdb7aafa7990e30bdf5
QSI-2026-18017	file_excerpt	src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp (line 497)	8f5dbe923034531e4f5a747ea0773d731c8847ed0736b3
QSI-2026-18017	file_excerpt	src/common/cosigner/eddsa_online_signing_service.cpp (line 382)	d641b422ac935789b7ebad5d737cd2cba4e16e40fddb2b
QSI-2026-18017	file_excerpt	src/common/cosigner/frost_cosigner_server.cpp (line 65)	2a70a07e1414d949a17e73ec9234ce658a7c844cbec200
QSI-2026-18016	file_excerpt	include/crypto/ed25519_algebra/ed25519_algebra.h (line 71)	5d36dd9af9202966201c370110f445cf52d9bd129264d9
QSI-2026-18016	file_excerpt	src/common/crypto/ed25519_algebra/ed25519_algebra.c (line 671)	e6eb5ecf7d33590258b8512c167df9e0bf49c6d9289b9b

Post-quantum assessment

	Key establishment (x = how long data must stay confidential; harvest now, decrypt later)	Signatures (x = how long signatures/ keys must stay trusted)
Data/key lifetime x	10	10
Migration time y	5	5
Assumed years to CRQC z	8 (early)	8
x + y − z	7 → migration would finish too late under this scenario	7 → migration would finish too late under this scenario

Planning assumptions: x for key establishment, x for signatures, y are the defaults of the risk profile “Crypto exchange, custodian or wallet”, not the client's figures; with the client's actual lifetimes the verdict can change.

Post-quantum maturity: Level 0 of 4 — no post-quantum cryptography

- Classical only — threshold signing and MPC protocols: ECDSA, Ed25519, FROST, Paillier, Pedersen commitment, Threshold ECDSA (45 file(s))

Next step: Inventory where classical public-key cryptography protects long-lived data and keys, and choose the first migration: usually a hybrid TLS key exchange and wrapped keys.

Levels: 0 none; 1 present, not wired; 2 in use outside the critical paths; 3 hybrid in a critical path; 4 post-quantum by default in the critical paths (root of trust, key wrapping, channel key exchange). Judged from code and manifests.

What the quantum-vulnerable algorithms do

Role	Algorithms (files)	ID	First step
threshold signing and MPC protocols	Paillier (30), Pedersen commitment (8), FROST (7), Threshold ECDSA (7), Ed25519 (3), ECDSA (1)	QSI-2026-18011, QSI-2026-18012, QSI-2026-18013, QSI-2026-18014, QSI-2026-18015, QSI-2026-18017	Track post-quantum threshold signatures (NIST IR 8214C) and keep the protocol layer ready to swap the signature scheme; record which keys and shares must stay valid for how long
role not determined	Ed25519 (2)	QSI-2026-18016	Classify each usage, record data lifetimes, and plan migration to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it; introduce an abstraction layer for crypto-agility

Roles are judged from file paths, function names and the code around each match; review confirms them. Root-of-trust keys and wrapped keys come first: they are hard to replace or can be recorded today.

z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Quantum status	Count
reduced margin (Grover), approved	1
Grover: acceptable to NIST, below ECCG/ANSSI guidance	1
vulnerable (Shor)	88

Detailed findings: QSI-2026-18011, QSI-2026-18012, QSI-2026-18013, QSI-2026-18014, QSI-2026-18015, QSI-2026-18017, QSI-2026-18016

PQC readiness questionnaire

No readiness questionnaire was completed for this project.

Quantum algorithms relevant to this inventory

Quantum algorithm	Effect	Inventoried algorithms (rows)	What it means for this audit
Shor's algorithm — factoring Q1: quantum computation offline, classical access to the victim	breaks	Paillier (30)	Breaks RSA at every key length: a longer modulus only raises the cost polynomially. Plan migration to ML-KEM (or a hybrid) for key establishment and ML-DSA / SLH-DSA for signatures; start with keys protecting data that must stay confidential for years (harvest now, decrypt later) and with long-lived signatures.
Shor's algorithm — discrete logarithms in finite fields Q1: quantum computation offline, classical access to the victim	breaks	Finite-field Diffie-Hellman (2)	Breaks finite-field Diffie–Hellman and DSA at every size. Move key establishment to ML-KEM (hybrid with X25519 or ECDH during the transition) and signatures to ML-DSA or SLH-DSA.
Shor's algorithm — elliptic-curve discrete logarithms Q1: quantum computation offline, classical access to the victim	breaks	ECDSA (15), Ed25519 (11), FROST (14), Pedersen commitment (8), Threshold ECDSA (7), X25519 (1)	Elliptic-curve keys are expected to fall before RSA keys of comparable classical strength. On blockchains, coins whose public key is already on chain (reused addresses, pay-to-public-key and Taproot outputs) are exposed first; plan the move to post-quantum signatures and avoid key reuse in the meantime.

Grover's algorithm — key search and preimages Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	HMAC (1), SHA-2 (1)	Not a break. 256-bit keys (AES-256, ChaCha20) keep a comfortable margin; AES-128 meets NIST category 1, while some authorities ask for 256-bit keys in quantum-resistant contexts (see the AES-128 entry). Prefer 256-bit keys for long-lived data; 256-bit hashes need no change for preimage resistance.
Brassard–Høyer–Tapp — collision search Q1: quantum computation offline, classical access to the victim	weakens (larger keys restore the margin)	SHA-2 (1)	256-bit hashes (SHA-256, SHA3-256) remain adequate for collision resistance; 160-bit hashes are already weak classically (SHA-1 collisions are practical). Use 384/512-bit hashes where NIST categories 4–5 are required.

Effects: breaks — replace the algorithm; weakens — larger keys or outputs restore the margin; included in standard parameters — no action at standard parameter sets; only with superposition queries — not applicable to ordinary deployments.

Limitations and assumptions

- This report presents automated and documented analysis results for the stated scope. It is not a legal opinion, a certification or an attestation of compliance with any standard.
- z is a planning assumption chosen per risk profile, not a prediction. A cryptographically relevant quantum computer is not publicly known to exist today.

Remediation plan

QSI-2026-18011 · P3 · 90 days

short-term risk reduction

Add Paillier usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18012 · P3 · 90 days

short-term risk reduction

Add Pedersen commitment usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18013 · P3 · 90 days

short-term risk reduction

Add Threshold ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18014 · P3 · 90 days

short-term risk reduction

Add FROST usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18015 · P3 · 90 days

short-term risk reduction

Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18017 · P3 · 90 days

short-term risk reduction

Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.

Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18016 · P3 · 90 days

short-term risk reduction

Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).

Rationale	Migration cannot be planned without knowing where and why the algorithm is used.
Expected outcome	Every usage has an owner, a lifetime and a migration path.
Complexity	medium
Dependencies	Access to system owners.
Side effects	None (documentation and design work).
Tests before change	Confirm the inventory entry matches the deployed configuration.
Tests after change	Inventory review signed off by the system owner.
Closure criteria	Inventory entry with owner, lifetime and target algorithm exists.

QSI-2026-18011 · P4 · 90 days

root-cause fix

Plan migration of Paillier to No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) or a hybrid scheme in use.

QSI-2026-18012 · P4 · 90 days

root-cause fix

Plan migration of Pedersen commitment to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.

Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows or a hybrid scheme in use.

QSI-2026-18013 · P4 · 90 days

root-cause fix

Plan migration of Threshold ECDSA to The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design or a hybrid scheme in use.

QSI-2026-18014 · P4 · 90 days

root-cause fix

Plan migration of FROST to As for the single-signer scheme it produces: ML-DSA or SLH-DSA where the verifier allows; threshold post-quantum signatures are still research (NIST IR 8214C) (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows As for the single-signer scheme it produces: ML-DSA or SLH-DSA where the verifier allows; threshold post-quantum signatures are still research (NIST IR 8214C) or a hybrid scheme in use.

QSI-2026-18015 · P4 · 90 days root-cause fix

Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-18017 · P4 · 90 days root-cause fix

Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

QSI-2026-18016 · P4 · 90 days root-cause fix

Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.

Rationale	Removes dependence on a quantum-vulnerable public-key primitive.
Expected outcome	The asset uses a NIST-standardized post-quantum or hybrid algorithm.
Complexity	high
Dependencies	Library/protocol support (e.g. OpenSSL ≥ 3.5, HSM firmware), interoperability with peers.
Side effects	Larger keys/signatures, possible performance and compatibility impact.
Tests before change	Interoperability tests in a staging environment.
Tests after change	Verify the negotiated/used algorithm on the production asset.
Closure criteria	

Re-audit shows ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/ composite form where the ecosystem supports it or a hybrid scheme in use.

Priorities for 30, 60 and 90 days

30 days

Priority	ID	Action	Type
None			

60 days

Priority	ID	Action	Type
None			

90 days

Priority	ID	Action	Type
P3	QSI-2026-18011	Add Paillier usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18012	Add Pedersen commitment usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18013	Add Threshold ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18014	Add FROST usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18015	Add ECDSA usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18017	Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P3	QSI-2026-18016	Add Ed25519 usages at source code to the cryptographic inventory with an owner and a data-lifetime estimate; design for crypto-agility (algorithm identifiers, configurable providers).	short-term risk reduction
P4	QSI-2026-18011	Plan migration of Paillier to No standardized post-quantum replacement. Plan threshold schemes over post-quantum signatures as they mature (NIST threshold call, IR 8214C), or protocols that avoid Paillier (oblivious-transfer based) (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18012	Plan migration of Pedersen commitment to (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

P4	QSI-2026-18013	Plan migration of Threshold ECDSA to The signature scheme is fixed by the blockchain or verifier. Track chain-level post-quantum proposals and threshold post-quantum signatures (NIST IR 8214C); keep key-share refresh and crypto-agility in the custody design (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18014	Plan migration of FROST to As for the single-signer scheme it produces: ML-DSA or SLH-DSA where the verifier allows; threshold post-quantum signatures are still research (NIST IR 8214C) (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18015	Plan migration of ECDSA to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18017	Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix
P4	QSI-2026-18016	Plan migration of Ed25519 to ML-DSA (FIPS 204) or SLH-DSA (FIPS 205), in a hybrid/composite form where the ecosystem supports it (or a hybrid scheme) once supported by your libraries, protocols and counterparties.	root-cause fix

Verification after remediation

ID	Retest procedure
QSI-2026-18011	Re-run the inventory after refactoring.
QSI-2026-18012	Re-run the inventory after refactoring.
QSI-2026-18013	Re-run the inventory after refactoring.
QSI-2026-18014	Re-run the inventory after refactoring.
QSI-2026-18015	Re-run the inventory after refactoring.
QSI-2026-18017	Re-run the inventory after refactoring.
QSI-2026-18016	Re-run the inventory after refactoring.

Tools used

Tool	Version
blockchain	1.0.0
certificates	1.2.0
chain_map	1.0.0
configs	1.1.0
dependencies	1.3.0
inventory_document	1.1.0
solidity	1.3.0
source_code	1.4.0
tls_code	1.1.0
tls_live	1.0.0
qsi-engine	1.3.0
OSV.dev API	v1

Sources and normative documents

Reviewed: 2026-10-13

1. A. Chailloux, M. Naya-Plasencia, A. Schrottenloher — An Efficient Quantum Collision Search Algorithm and Implications on Symmetric Cryptography (ASIACRYPT 2017; IACR ePrint 2017/847) — IACR ePrint. https://eprint.iacr.org/2017/847 (peer-reviewed article: Õ(2^{2n/5}) time, O(n) qubits, Õ(2^{n/5}) classical memory)
2. C. Chevignard, P.-A. Fouque, A. Schrottenloher — Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves (IACR ePrint 2026/280) — IACR ePrint. https://eprint.iacr.org/2026/280 (revision of a EUROCRYPT 2026 paper: 3.12n + o(n) qubits; for n = 256, 1193 qubits and 22 runs of 2^38.98 Toffoli gates)
3. C. Gidney — How to factor 2048 bit RSA integers with less than a million noisy qubits (arXiv:2505.15917) — arXiv. https://arxiv.org/abs/2505.15917 (preprint (2025-05-21): < 1 million noisy qubits, < 1 week, 0.1% gate error, 1 µs surface-code cycle)
4. C. Gidney, M. Ekerå — How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits (Quantum 5, 433) — Quantum. https://doi.org/10.22331/q-2021-04-15-433 (peer-reviewed article (2021): 10^-3 gate error, 1 µs cycle, 10 µs reaction time)
5. C. Zalka — Grover's quantum searching algorithm is optimal (Phys. Rev. A 60, 2746, 1999; arXiv:quant-ph/9711070) — APS. https://doi.org/10.1103/PhysRevA.60.2746 (peer-reviewed article: optimal for any success probability; parallel search gains no more than splitting the space)
6. Common Vulnerability Scoring System v3.1: Specification Document — FIRST. https://www.first.org/cvss/v3.1/specification-document (final)
7. D. J. Bernstein — Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? (2009) — cr.yp.to. https://cr.yp.to/hash/collisioncost-20090823.pdf (paper dated 2009-08-23)
8. D. Litinski — How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates (arXiv:2306.08585) — arXiv. https://arxiv.org/abs/2306.08585 (preprint (2023-06-14): 6000 modules × 1152 physical qubits, one key per 10 minutes under its architecture assumptions)
9. FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM) — NIST. https://csrc.nist.gov/pubs/fips/203/final (final (August 2024))
10. FIPS 204 — Module-Lattice-Based Digital Signature Standard (ML-DSA) — NIST. https://csrc.nist.gov/pubs/fips/204/final (final (August 2024))
11. FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA) — NIST. https://csrc.nist.gov/pubs/fips/205/final (final (August 2024))
12. G. Brassard, P. Høyer, A. Tapp — Quantum Algorithm for the Collision Problem (LATIN '98; arXiv:quant-ph/9705002) — Springer / arXiv. https://arxiv.org/abs/quant-ph/9705002 (peer-reviewed article: collisions of r-to-one functions in O((N/r)^{1/3}) expected evaluations)
13. J. Proos, C. Zalka — Shor's discrete logarithm quantum algorithm for elliptic curves (Quantum Inf. Comput. 3(4), 2003; arXiv:quant-ph/0301141) — Quantum Information & Computation. https://doi.org/10.26421/QIC3.4-3 (peer-reviewed article)
14. L. K. Grover — A fast quantum mechanical algorithm for database search (STOC 1996) — ACM. https://doi.org/10.1145/237814.237866 (peer-reviewed article)
15. M. Amy et al. — Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (SAC 2016; arXiv:1603.09383) — Springer / arXiv. https://arxiv.org/abs/1603.09383 (peer-reviewed article: SHA-256 about 2^153.8 surface-code cycles on 2^12.6 logical qubits; SHA3-256 about 2^146.5 cycles on 2^20)
16. M. Cain et al. — Shor's algorithm is possible with as few as 10,000 reconfigurable atomic qubits (arXiv:2603.28627) — arXiv. https://arxiv.org/abs/2603.28627 (preprint (2026-03-30): P-256 in a few days with 26,000 physical neutral-atom qubits; RSA-2048 one to two orders of magnitude longer)
17. M. Ekerå, J. Gärtner — A high-level comparison of state-of-the-art quantum algorithms for breaking asymmetric cryptography (arXiv:2405.14381) — arXiv. https://arxiv.org/abs/2405.14381 (preprint (2024-05-23): Regev-style algorithms show no overall advantage yet for cryptographic RSA and finite-field sizes)
18. M. Mosca — Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy, 2018) — IEEE. https://doi.org/10.1109/MSP.2018.3761723 (peer-reviewed article; source of the x + y > z migration inequality)
19. M. Roetteler, M. Naehrig, K. Svore, K. Lauter — Quantum resource estimates for computing elliptic curve discrete logarithms (ASIACRYPT 2017) — IACR / arXiv. https://arxiv.org/abs/1706.06752 (peer-reviewed article: at most 9n + 2⌈log2 n⌉ + 10 qubits and 448 n^3 log2 n + 4090 n^3 Toffoli gates)
20. NIST IR 8547 — Transition to Post-Quantum Cryptography Standards — NIST. https://csrc.nist.gov/pubs/ir/8547/final (final (2026-10-27; the initial public draft of November 2024 is obsoleted). Tables 2 and 4: ECDSA, EdDSA and RSA signatures, finite-field and elliptic-curve DH and MQV, and RSA key establishment are deprecated after 2030 and disallowed after 2035 at every key size — the draft deprecated only the 112-bit level. A composite (hybrid) key establishment stays FIPS

140-validatable while one input comes from an approved scheme. Repeats the federal deadlines of OMB M-26-15: post-quantum key establishment by 2030-12-31 and signatures by 2031-12-31 for high value assets and high impact systems)

21. NIST — Post-Quantum Cryptography Standardization: Evaluation Criteria, Security (Call for Proposals, Section 4.A.5) — NIST. https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization/Evaluation-Criteria/Security-(Evaluation-Criteria) (security categories: AES-128 key search 2^170/MAXDEPTH quantum gates, AES-192 2^233/MAXDEPTH, AES-256 2^298/MAXDEPTH; MAXDEPTH 2^40–2^96)
22. P. W. Shor — Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer (SIAM J. Comput., 1997) — SIAM. https://doi.org/10.1137/S0097539795293172 (peer-reviewed article)
23. R. Babbush et al. — Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations (arXiv:2603.28846) — arXiv. https://arxiv.org/abs/2603.28846 (preprint (2026-03-30, v2 2026-04-15): 256-bit ECDLP with < 1200 logical qubits and < 90 M Toffoli gates (or < 1450 and < 70 M); < 500,000 physical qubits at 10^-3 error, minutes)
24. S. Jaques, M. Naehrig, M. Roetteler, F. Virdia — Implementing Grover oracles for quantum key search on AES and LowMC (EUROCRYPT 2020; arXiv:1910.01700) — Springer / arXiv. https://arxiv.org/abs/1910.01700 (peer-reviewed article: lower cost estimates for the AES key searches that define the NIST categories)

Appendices

A. Cryptographic inventory

Algorithm	Parameter	Quantum status	Classical status	Count	Locations	Examples
HMAC		reduced margin (Grover), approved	acceptable	1	1	src/common/blockchain/mpc/hd_derive.cpp:37
SHA-2		Grover: acceptable to NIST, below ECCG/ANSSI guidance	acceptable	1	1	src/common/blockchain/mpc/hd_derive.cpp:37
ECDSA		vulnerable (Shor)	acceptable	13	13	include/cosigner/bam_ecdsa_cosigner.h; include/cosigner/bam_ecdsa_cosigner_client.h; include/cosigner/bam_ecdsa_cosigner_server.h
ECDSA	secp256k1	vulnerable (Shor)	acceptable	1	1	include/cosigner/sign_algorithm.h:10
ECDSA	P-256	vulnerable (Shor)	acceptable	1	1	include/cosigner/sign_algorithm.h:11
Ed25519		vulnerable (Shor)	acceptable	6	6	include/cosigner/asymmetric_eddsa_cosigner.h; include/cosigner/asymmetric_eddsa_cosigner_client.h; include/cosigner/asymmetric_eddsa_cosigner_server.h
Ed25519	Ed25519	vulnerable (Shor)	acceptable	5	5	include/crypto/ed25519_algebra/ed25519_algebra.h:71; src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp:401; src/common/cosigner/eddsa_online_signing_service.cpp:382
FROST		vulnerable (Shor)	acceptable	14	14	include/cosigner/frost_cosigner.h:17; include/cosigner/frost_cosigner_client.h:3; include/cosigner/frost_cosigner_server.h:3
Finite-field Diffie-Hellman		vulnerable (Shor)	acceptable	2	2	include/crypto/zero_knowledge_proof/diffie_hellman_log.h; src/common/crypto/zero_knowledge_proof/diffie_hellman_log.c
Paillier		vulnerable (Shor)	acceptable	30	30	include/cosigner/bam_ecdsa_cosigner.h:59; include/cosigner/bam_ecdsa_cosigner_client.h:118; include/cosigner/bam_ecdsa_cosigner_server.h:97

Pedersen commitment	vulnerable (Shor)	acceptable	8	8	include/cosigner/bam_ecdsa_cosigner_client.h:8; include/cosigner/bam_key_persistency_structures.h:20; include/crypto/commitments/pedersen.h:25
Threshold ECDSA	vulnerable (Shor)	acceptable	7	7	include/cosigner/cmp_ecdsa_offline_signing_service.h:5; include/cosigner/cmp_ecdsa_online_signing_service.h:5; include/cosigner/cmp_ecdsa_signing_service.h:90
X25519	vulnerable (Shor)	acceptable	1	1	src/common/crypto/ed25519_algebra/curve25519.c

Long lists are grouped here; the full per-file list is in the JSON report, signed with the same key.

B. JSON

qsi.report/1 · SHA-256 23fbd77d28cfee88fe35b18ec32795741bb197e17d217cbdef227c02036a02e3

C. Input files

File	Size	SHA-256
include/blockchain/mpc/hd_derive.h	2460	0784ee83beae49d092d35b9db7d53473fefc540c8be442909872c430e86caf78
include/cosigner/asymmetric_eddsa_cosigner.h	1726	169950866b4e92cbfd9d3a42f4d936cc1b412754fe0baebcc50cef0d076670a7
include/cosigner/asymmetric_eddsa_cosigner_client.h	2218	16731f4562f67388faac8571963ce2ef190c743c766215ba0e6447e630490f86
include/cosigner/asymmetric_eddsa_cosigner_server.h	4082	5470761c6a191df6d35066e9515e9b75ee2d252693db9d6fc6d907b0590091e5
include/cosigner/bam_ecdsa_cosigner.h	14971	43a3eaafe7efc5873b9ad1c3f9e1adf530817a9afb8c7957d95bee435791e002
include/cosigner/bam_ecdsa_cosigner_client.h	7693	bbb3628a1a863822935009bbef8e29f3d0365bd91d6f7e29a8cd9888d00a596c
include/cosigner/bam_ecdsa_cosigner_server.h	6136	1f878ae3e3f65f63928b4c355ab773793c9ca8f6f7470ddcfaf7f1e0adcf0ccf
include/cosigner/bam_key_persistency_client.h	822	d73ee8f07242da054391559bcb7038946cb6316e003390579c4ddb7d77a2536f
include/cosigner/bam_key_persistency_common.h	1063	1cac38ce587bb9feee0f8b6273367aece8df19ab680d88af33551f99fbc382bd
include/cosigner/bam_key_persistency_server.h	1031	6618e20c1d07432c59cecb957e1856b39de444e2917b21ecfa5e410d973cdfb3
include/cosigner/bam_key_persistency_structures.h	1870	b8f440b5865c6694f9ba426c3d8168e5168e9ee5253c071aa1536566caaf36e7
include/cosigner/bam_key_persistency_structures_client.h	1153	7cbd61de453cb52fb93e21b68e44f46e50e0826fa98faed3efb02a1f9c603dfa

include/cosigner/bam_key_persistency_structures_server.h	824	68e9a88bb24c8408be5bf6de351772ed4d89cd0e2e83b0defbf740061d2b1e1d
include/cosigner/bam_tx_persistency_client.h	543	461beec4f4e4e5ac6d65f106a5864893c077d94fd9798c64c89a91b88ec7426c
include/cosigner/bam_tx_persistency_common.h	281	461657be8121da86d246e4241fda46a0cb986f4e7b812f54b5a5ee86fd5888ea
include/cosigner/bam_tx_persistency_server.h	591	2ad3586e46df73362dc95d01207932313fea3fbd775cf6ad7b1ddbd9c121b398
include/cosigner/bam_tx_persistency_structures.h	1397	44834e5bdadde1ebd6d6eb985975ec7fe18f2b892e690729b4a186619149b386
include/cosigner/bam_tx_persistency_structures_client.h	274	edbfbb0312447a9f4742ad4b4fbd034401644ff7502463bbebf18fb6cdafd810
include/cosigner/bam_tx_persistency_structures_server.h	636	b7179096be2c13a5a6a2b0cda14e8ede57f7f5133aa1ee78052ec29304ebeea9
include/cosigner/cmp_ecdsa_offline_signing_service.h	3823	5f6e2ebfef941b6d5fcbaec28e9166686cb6bd90a9e50f1abd4f942632a482f3
include/cosigner/cmp_ecdsa_online_signing_service.h	2515	88a9477e41b6be37e08d23543670a2617bfe0aac058e160f635c5c1efe8dab14
include/cosigner/cmp_ecdsa_signing_service.h	7579	5f9cb66f067166d12e7f344c95b27bcb5591daac3fc218da8ef728e9eea5f2ad
include/cosigner/cmp_key_persistency.h	2242	a77659e68304ced274c756d459a7e1f3d127ec757e64475a4f7354c6e9200662
include/cosigner/cmp_offline_refresh_service.h	3008	fda35dbade09312978d49a10e670c1af8919e02afd249f620ce5176864913b1c
include/cosigner/cmp_setup_service.h	7659	6aca708a75069644771ee35eace129cefb209595d2d2bc1c154bb0afa5adc459
include/cosigner/cmp_signature_preprocessed_data.h	245	bc2e5815e989a5764b86e0fcee5759e2033f3a62ec7db0d6e7a7675fd640e004
include/cosigner/cosigner_exception.h	6049	cef299006c368984a7d0d16f02683c8b76f8ef0b7912caedad1387315d3c2430
include/cosigner/cosigner_status.h	440	8b8350ea3e186510e82249079ab6bf947a5ea81dd4d96a64aa8af33904c672d6
include/cosigner/eddsa_online_signing_service.h	3082	8dd11c70c530ddfac01863aecd07c1fd4b0b233ffbfd038a810d941362c3b575
include/cosigner/frost_cosigner.h	6382	cdfdfc52b15629df12a50a1b341ca816e02862988ac659b8be302c190fc779a1
include/cosigner/frost_cosigner_client.h	2552	1fff442d632f1c26700d7c6ec8f1a48b0c2eb2cf529a4b48358b12916eb93b00
include/cosigner/frost_cosigner_server.h	3201	423f364293a16ec6fb0bb45aa1f55c02e504a41c11b32015f2afd9a7de068a9b
include/cosigner/frost_key_persistency_client.h	806	ff8b8509b0137a680e957ac021f64bacd38bbd7392e883a78140553f87b71620
include/cosigner/frost_key_persistency_common.h	908	b71823c6bff5dad2b13169830518b7d6365f3b29eb8097ba55976e4d54ce3109
include/cosigner/frost_key_persistency_server.h	586	bd4ceafc5d5d9259ae67578b8c2cf77dff9974eebe167be292f01c6b3f1141f9

include/cosigner/frost_key_persistency_structures.h	855	0e42750d6967f8781422d6615ae72a70e6f28a8c4f25eda1359bfa3a88f98f6f
include/cosigner/frost_key_persistency_structures_server.h	347	080fd2c6afad4f86fca21157059023a3e40cb321fa4ae2b1c87bb73782de1d8d
include/cosigner/frost_tx_persistency.h	587	2242110aadb438d3711dab4eef10fc90a8cfac8cd78ac3512e4ac54464332e71
include/cosigner/frost_tx_persistency_structures.h	1102	2bafbb8ec522e43234c180ff90a987f2264fd11d82d522e68d4420115cd02646
include/cosigner/key_persistency_base.h	426	8283effc85fe9e9761e7ca25348dda403840e9b52218865fc3271983d306b250
include/cosigner/mpc_globals.h	1120	444ae84b5a7e84c6d87573b08d58e574fd462d8d8c12985d75e49996c2399439
include/cosigner/platform_service.h	4101	9c7e53a65dd5bc6b1f23c3b967fbb7bc6c993089b03f0d53cf73c373dd46b719
include/cosigner/prf.h	679	2ed02b703ce8f3ebdfcebb854af5c4b5848fcff2b91533bec23a8d03aa8c27d3
include/cosigner/sign_algorithm.h	353	99f94f61f6a92527b18600cbda1f670165e5c81277ec57e2665b4f04569b2c7f
include/cosigner/timing_map.h	864	c6a3ce8f6ddfe37bff38d67e4ef9a88b86a8ff20c68ccf28b5d0c59463c71505
include/cosigner/types.h	3922	04948c86393a4bb19bfb369ae866ec6cf3bc86891daf5e7dd3cb21fbd9be7742
include/crypto/GFp_curve_algebra/GFp_curve_algebra.h	5121	49913236e60e6700aac726b9c63d302df090489e6dd4aa5426fd30060daf8858
include/crypto/algebra_utils/algebra_utils.h	11211	9a1ffa32af76235e1f77895c494b8d2bc303517c3c489298ffe0d07d30cf0ac2
include/crypto/algebra_utils/status_convert.h	809	9201af8b1a40ffc321541c1d79e8b2bfa0d84bd156f955600351db111fc2f314
include/crypto/commitments/commitments.h	2418	bafa5bfe2997446e8b07ca7cf8d60901437c814a7dca3eef76a69ea0d9f8f735
include/crypto/commitments/damgard_fujisaki.h	8178	b9601697d3d1695089f9e69bb0c75bcf4b2af6b53010e2d0b5d1d86936cf8f14
include/crypto/commitments/pedersen.h	5971	64f9b6334199aea590507c2d5dab9c695bf8069f3d893f92d478a349ca88c393
include/crypto/commitments/ring_pedersen.h	3200	0b4dc766259c76e181692e5fa2279f32664088ec439821b72d6a253b8452d758
include/crypto/common/byteswap.h	222	ef3cab9dcfd0632690ecd2c3f0a861750ed1cbfebbac72150f4cb1ab86c7be41
include/crypto/drng/drng.h	903	18ba32d532f6673e8165892241a597046ca5a81289f09ff206f926ac88edec43
include/crypto/ed25519_algebra/ed25519_algebra.h	6034	f2e14ae6c9654a17d94401fedf48d44faf89f98bbb7fa3ade977d78e16c859f5
include/crypto/elliptic_curve_algebra/elliptic_curve256_algebra.h	6397	7837540fe3fbc5403ea2b238e8cde680376ab52bb0f1cc6cf4c4c923a814031e
include/crypto/elliptic_curve_algebra/elliptic_curve_algebra_status.h	630	ab229f739ccbeacfae1b11618ba8f4cf985719f0cfeaf2367cb2a5a746710345
include/crypto/keccak1600/keccak1600.h	1020	a9c972249a1afbbb03250fa489ff08e067802fad241af9c4f703002223e416a9
include/crypto/paillier/paillier.h	6215	c0f1722b6c676d850e623718173092ee2d0e3eeb3b3c4ef367ba02d3990b6041

include/crypto/paillier_commitment/paillier_commitment.h	16150	7ca49e7df7ab6a5e4471a37aaf16528462f29812b1a288a7c9145bbbf8c69595
include/crypto/shamir_secret_sharing/verifiable_secret_sharing.h	5534	41887a87d29f7f7c5479a9708105cf6f634dbee6dde59b4a3464f16256949e11
include/crypto/zero_knowledge_proof/diffie_hellman_log.h	1767	91547a7a19d8f34793230998cb50f995ed55498833de4eeb2269631dfff6332e
include/crypto/zero_knowledge_proof/range_proofs.h	21986	3fc59ce86e1b596a686bf5a5d1807183c6a8927e95fb6ed9bb84f18c338ff505
include/crypto/zero_knowledge_proof/schnorr.h	1972	2ff4a6896a51f22ab94e2a92016d3a379c32602327d713625ad55be4a34eae4d
include/crypto/zero_knowledge_proof/zero_knowledge_proof_status.h	391	b9cadc821ac263b455766721ea1eadebe4092c5604929ccd912bfbd6fe2df33d
include/logging/logging_t.h	1421	c30876d049ea7d0cfcdbe975963ceb338cf6c3d04a879b580cddba6344616cb9
include/utils/string_utils.h	828	692a33d6398c3ae1a0087bc3a84bb7a693947451aa8ad54a9268ad66e0ce5d0e
src/common/blockchain/mpc/hd_derive.cpp	6573	9eb0cd525cf9d789d344dabf4d018f4cf6bbe068469de06df4633038c8656f0e
src/common/cosigner/asymmetric_eddsa_cosigner.cpp	3250	28792e6b01354a26246089f7ed5a566627318af5d019b2256d9e048fb63ff029
src/common/cosigner/asymmetric_eddsa_cosigner_client.cpp	9115	3bd187f3c497261687848b8c2007d4e9e110382ad9b89134b7af8ae0784987df
src/common/cosigner/asymmetric_eddsa_cosigner_server.cpp	23567	15e49277a9de05d894ab60fec551a03a165cbf819503c0ef62c9731a8e940510
src/common/cosigner/bam_ecdsa_cosigner.cpp	16006	c786bd2dd59551293f2a1b0f63e8ceaf00fad63164111b2e4a8a1afe36918211
src/common/cosigner/bam_ecdsa_cosigner_client.cpp	42102	d5e5a60b90b9d16e7f9259a789c48b3da7b09b952486f642ac71c89d13928846
src/common/cosigner/bam_ecdsa_cosigner_server.cpp	52622	dfa91f27d28fe42406ad9f82703190fa39117aaaf8251409f585104d21c253dc
src/common/cosigner/bam_key_persistency_structures.cpp	566	bd29f7359d3d244d8dd06e234ae24061a873909e617d1fb0a1089af38eaa437f
src/common/cosigner/bam_well_formed_proof.cpp	21340	e7393aba28ea97975f47837f376d0c6a71207cdc494d430110cb31905cd9d6a1
src/common/cosigner/bam_well_formed_proof.h	2605	9134bd9ccc2c46c137454e472d6d75e5b1b7ecf24e5db371e6e886e9446390cb
src/common/cosigner/cmp_ecdsa_offline_signing_service.cpp	24142	29718caacb308f10d62e5499870b165ab530eb41f3ae28e45961760d5c0c5ed2
src/common/cosigner/cmp_ecdsa_online_signing_service.cpp	23723	306e2d513a68a91767c6ea130f0656bc0f2f5fe2a434e1f4ea489759458ad08d
src/common/cosigner/cmp_ecdsa_signing_service.cpp	17616	5439ad96d0f527698e80ec2ac3f5cb79de30473030a3c286dcfbad086c11e08f

src/common/cosigner/cmp_key_persistency.cpp	113	d110dbf7b2b107270ae888b199778278ca431c8fca15daf7cbaad272aa19e3f4
src/common/cosigner/cmp_offline_refresh_service.cpp	12803	5dd2c7be321e626f46654e85548e37a260b523825dc16db90a6e22b6fbe03247
src/common/cosigner/cmp_setup_service.cpp	48451	6deff1ddfe05794f32893b1fb10d16e5a3bccce52ee7eb943bf4567197b5369d
src/common/cosigner/cosigner_bn.h	1924	f208924fac4359214cb5b6d568d987a8e933095fde9323e4907d40bb97554ca7
src/common/cosigner/cosigner_exception.cpp	6641	e482b9f0cd4c2330ffe6a4016eb8f45d9e6d8d5f2a8ac2dd6aaee23960b0fa6a
src/common/cosigner/eddsa_online_signing_service.cpp	20543	e8264a8e3e84d0925f268fb503668164566314721d12e7add8fa3c9cf6956585
src/common/cosigner/frost_cosigner.cpp	17691	18d70531bf58d27872d80c165073ccd86e961cb6413057cf03c16955ce42434f
src/common/cosigner/frost_cosigner_client.cpp	15740	552f862baf1eb9a1cc2c3cea76ced86fe677ee6710e2df336fdd6d06532f7b67
src/common/cosigner/frost_cosigner_server.cpp	23226	a356cd59f4b11d71d5c389462a1cd5335f82f73e1c2ba76e89f075d880fe8a59
src/common/cosigner/mta.cpp	71972	af4fcb380b24ab2182617e9125fafcf87bb93dca10105af656639e1f79752439
src/common/cosigner/mta.h	7447	c87c5e80f289596142d2aa5825ca8aa6af1f65bf4c99280772f9983e87b8d75d
src/common/cosigner/platform_service.cpp	153	253ccbadae3c4312d7004b287f8768f2a3cae7709b645526e356026357d0a08f
src/common/cosigner/timing_map.cpp	1238	2727160df7f638ed7cd1cf20f54e7103c45baf68fc678cda846b69bbccb7146e
src/common/cosigner/utils.cpp	7914	5853899526a8b45bd2b3bfe4620e55998112a66e9dd19a4eae4b1bf35b65cff1
src/common/cosigner/utils.h	4222	7edebde9a5d263d0aee958ca897d8672a1d1990ef371fc238f746429b9423060
src/common/crypto/GFp_curve_algebra/GFp_curve_algebra.c	48357	4b3362de3176d745651ad78179e50d5c74f9237f0f60b2ef47b950c48eb5087d
src/common/crypto/algebra_utils/algebra_utils.c	12642	c910a2390d1ca5a52c203da3c7c8938b0d06e51c030468a38717d1b9c20f6dbb
src/common/crypto/algebra_utils/status_convert.c	4010	0ae771e97b2d1bfd951abb81ad08a92e2d0969c1c6dd16c761965c03d43368e1
src/common/crypto/commitments/commitments.c	4458	0eb46451d67820b7bc565013d9373a2c8124ccb68555327224bb39c633ef2a9e
src/common/crypto/commitments/damgard_fujisaki.c	23422	76516911357fec5af9eb16d03e12fc9f0edc50f99307cef577f84351a3e0d32a
src/common/crypto/commitments/damgard_fujisaki_internal.h	1837	d8534c63168d5e865fc8d830e60012ac7ec7286b21997b957d3735a589f0921a
src/common/crypto/commitments/pedersen.c	5776	6937e55e6449d638d8d2f957e2ef5842b580880e516a2c659a286b045ade27d4
src/common/crypto/commitments/ring_pedersen.c	30493	98df1169e7501fc8914529d39c2a9ca3f578c2efa58e785e75b6bc8bacf3d09b

src/common/crypto/commitments/ring_pedersen_internal.h	1011	fe982492ed3153ec27bf490e00d32d1a6393dd22d3e719f147f5a6305f486a4d
src/common/crypto/drng/drng.c	1920	6a18405f24fa5aec884307e46dfc7bf4f4a58d0c33584366645a1d8e63b15b24
src/common/crypto/ed25519_algebra/curve25519.c	201636	65798d86db94c7c039624d90ce5cc7e10deca9a154d72f76a39cc6071624a4c9
src/common/crypto/ed25519_algebra/ec_lcl.h	347	f2b9c81c236f7897892ce255551dbbdce71998800ae92bc219097982f486635f
src/common/crypto/ed25519_algebra/ed25519_algebra.c	40091	f772f412431b5cd19b6a98ce6ca6211c53ed71bdaac46e2f1d620e4317ff39b3
src/common/crypto/keccak1600/keccak1600.c	2213	d4a274860c749266c5e83305792235759820b2ea5d1186e88baf951ed181dbca
src/common/crypto/paillier/alpha.h	72623	871214587e980e406a0d39cbd9bd95342c76a135df40156b0baa7f68055091e6
src/common/crypto/paillier/paillier.c	40132	8d7ac9b1485a4af5941dc477589b1dfad4a3411522907e12464aaedf078e342a
src/common/crypto/paillier/paillier_internal.h	1170	19357679842b6a9003874e35cd2ed585a9ef1dd969a8d29efc09810a9098330d
src/common/crypto/paillier/paillier_zkp.c	41704	fba4e86186788ed24d1f980fd86d8239fd9754c3153efc57389c7b3d807c9846
src/common/crypto/paillier_commitment/paillier_commitment.c	56367	1ce881799a80a849419d30e9f9954ec121e8196a068787ced0453cfe650778a1
src/common/crypto/paillier_commitment/paillier_commitment_internal.h	3955	022b54db4c7ea973cb11a5e6d8cdc7909ce2b39e6a33ab092d60916057764e00
src/common/crypto/shamir_secret_sharing/verifiable_secret_sharing.c	21183	6ffd1bb550069ece8673722e6da7e972d28eac3f1bf0fc44b4cb31a9946145a6
src/common/crypto/zero_knowledge_proof/damgard_fujisaki_zkp.c	20614	86b257f5c2c7cf380708f1b2bb88234faecc8a3f53f51c7530e9f2778940247d
src/common/crypto/zero_knowledge_proof/diffie_hellman_log.c	8851	bb286960abd2c00e0cc31883eb4d0b23b9b5986a54cc345796f69b598f69f99c
src/common/crypto/zero_knowledge_proof/range_proofs.c	119448	a248bcb8413e65b2baa58a8fe5f04b2306c46f7d15b85966f201d3b9d1c8c6d5
src/common/crypto/zero_knowledge_proof/schnorr.c	7615	97e3d051dddf103a6712afe6f920d490f908ad528205cd905a2d19857eae9696
src/common/crypto/zero_knowledge_proof/zkp_constants_internal.h	1369	0487839ea08aff34c821d69d2c39f61f29a123a2cccadd277ef12be7e7ea3eb6
src/common/logging/logging_t.c	1123	2270f230ddc2e1959c7d3f44d241c2a41b7bc56f722c30c0680a66719a0fd12d